

多模态室内交互系统五天项目规划

1. 项目背景

原项目以艺术导览为目标场景，但该场景对数据、知识库、训练与验证条件要求较高，不利于在五天内完成稳定闭环。因此本阶段将场景调整为更通用、可复现、易验证的实验室场景。

本阶段重点不是追求完整的大规模多 Agent 系统，而是在五天内完成一个可演示、可验证、可扩展的多模态交互系统原型，为后续 Agent 化演进打下基础。

2. 本阶段目标

在五天内完成一个可运行的多模态交互系统，具备以下能力：

- 基于 YOLO 的目标检测与指向目标定位
- 基于 DeepSeek VLM 的视觉理解、视觉问答与最终回答生成
- 基于规则的路由器，用于验证多链路协同流程
- 基于工具调用的最小 Agent，用于替代规则路由并完成基础调度

3. 目标场景选择

3.1 场景切换原则

本阶段不再采用艺术导览场景，而是统一采用实验室场景进行开发和验证。

选择实验室场景的原因如下：

- 目标物容易获取，采样成本低
- 背景相对可控，便于重复测试
- 通用 YOLO 类别覆盖更高，减少额外训练压力
- VLM 对日常物体描述更稳定
- DeepSeek VLM 可直接完成视觉理解与文本生成，便于缩短链路

3.2 推荐目标物

建议优先围绕以下类别构建样例：

- cup
- bottle
- mouse
- keyboard
- remote
- scissors
- lamp
- box
- notebook
- phone
- tool
- container

如时间允许，可增加少量实验室特色物品，但本阶段不建议为了类别扩展引入额外训练任务。

4. 系统能力设计

本阶段系统包含三类核心回答链路。

4.1 纯语言链路

适用于非视觉问题、普通知识问答和简单闲聊。

处理流程：

ASR -> DeepSeek VLM(纯文本模式) -> TTS

4.2 视觉直答链路

适用于简单视觉问题，例如

- 这是什么颜色
- 这里有什么
- 我前面有什么东西

处理流程：

ASR + image/crop -> DeepSeek VLM -> TTS

4.3 感知增强回答链路

适用于带有指向、定位和解释需求的问题，例如：

- 我指的这个是什么
- 介绍一下这个
- 这个和旁边那个有什么区别

处理流程：

hand_stream -> YOLO -> crop/上下文 -> DeepSeek VLM -> TTS

5. 技术路线

5.1 第一阶段：规则路由器验证

先采用规则路由器对用户输入进行判断，验证以下能力是否能够稳定协同：

- 是否需要视觉输入
- 是否需要指向上下文
- 是否直接调用 DeepSeek VLM
- 是否需要结合 YOLO 指向上下文后再调用 DeepSeek VLM

该阶段目标是尽快打通主流程，明确模块边界、接口格式和延迟瓶颈。

5.2 第二阶段：最小 Agent 接管

在规则路由器稳定后，将检测与视觉理解模块封装为工具，由 Agent 负责按问题类型选择调用顺序。

该阶段的 Agent 目标不是复杂规划，而是验证以下能力：

- 识别当前问题类型
- 决定调用哪些工具
- 按固定顺序组织一次完整回答

因此本阶段 Agent 以最小闭环为目标，不追求复杂任务规划能力。

6. 模块职责划分

6.1 感知模块

负责：

- 手势检测
- 目标检测
- 指向命中判断
- 目标框与图像区域输出
- 结构化视觉上下文生成

当前项目已具备感知模块基础，相关代码位于 `services/hand_stream.py`。本阶段重点不是从零实现感知能力，而是基于现有 `hand_stream.py` 调整场景适配、目标物输出、结构化结果格式以及与后续 DeepSeek VLM 链路的接口。

建议输出统一结构化结果，例如：

```
{
  "timestamp": 0,
  "has_hand": true,
  "pointing": true,
  "target_class": "cup",
  "bbox": [120, 80, 300, 260],
  "confidence": 0.81,
  "frame_path": "rviz_captured_images/latest.jpg"
}
```

6.2 视觉理解模块

负责：

- 对整图或目标区域进行视觉描述
- 回答简单视觉问题
- 直接生成最终回答

建议优先使用 DeepSeek VLM 同时完成视觉理解与回答生成，减少多模型串联带来的额外延迟和工程复杂度。

6.3 回答生成模块

负责：

- 接收用户问题
- 结合视觉区域与指向上下文信息
- 直接生成最终自然语言回答
- 保持回答风格一致性

6.4 路由与 Agent 模块

负责：

- 根据问题类型选择链路
- 决定是否调用 YOLO
- 决定是否调用 DeepSeek VLM
- 决定是否需要结合 YOLO 上下文一起送入 DeepSeek VLM

7. 人员分工建议

团队共 4 人，建议按以下方式分工。

7.1 物品识别与感知负责人 1人

负责内容：

- 实验室场景确认与目标物清单确认
- 基于 `services/hand_stream.py` 调整 YOLO 检测与 `hand_stream` 输出
- 指向命中稳定性验证
- `bbox` 和 `crop` 输出接口

交付物：

- 实验室场景样例数据
- 结构化感知输出
- 指向识别测试结果

7.2 路由器与 Agent 负责人 1到2人

负责内容：

- 规则路由器设计与实现
- DeepSeek VLM 接入
- 视觉上下文与 DeepSeek VLM 提示拼接整合
- Agent 工具封装与联调

交付物：

- 路由器版本
- 最小 Agent 版本
- 三链路联调结果

8. 五天任务安排

第 1 天

完成以下工作：

- 基于 `services/hand_stream.py` 调整感知模块输出结构
- DeepSeek VLM 接口接入与单独调用验证
- 路由模块完成基础骨架

第 2 天

完成以下验证：

- YOLO -> 指向上下文 输出稳定
- VLM 单独调用可用
- 简单视觉问题可直接回答

第 3 天

打通主链路：

- YOLO -> DeepSeek VLM
- 完成“指向某物并提问”的完整回答闭环

第 4 天

完成以下任务：

- 三类问题规则路由验证

第 5 天

完成以下任务：

- 接入最小 Agent
- 验证工具选择流程
- 与规则路由结果进行对比
- 系统联调
- 日志与异常处理补齐
- 指标整理
- 演示脚本与汇报材料准备

9. 验证指标

本阶段重点关注以下指标：

- 指向目标识别成功率
- 简单视觉问题回答正确率
- 指向场景下回答相关性
- 规则路由成功率
- Agent 工具选择成功率

建议至少准备以下测试样例：

- 20 个视觉问答样例
- 20 个指向识别样例

10. 风险与应对

本阶段主要风险包括：

- 指向目标不稳定
- VLM 响应时间过长
- YOLO 类别与目标场景不完全匹配
- DeepSeek VLM 未能稳定利用指向上下文

对应措施如下：

- 使用固定桌面场景降低环境干扰
- VLM 优先处理 bbox crop 而不是整图

- 本阶段优先使用通用类别，避免引入额外训练压力
- 明确视觉上下文 prompt 模板

11. 本阶段交付物

本阶段最终交付内容包括：

- 可运行的多模态 demo
- 规则路由器版本
- 最小 Agent 版本
- 测试样例与指标统计表
- 项目总结与后续迭代建议

12. 后续演进建议

本阶段完成后，后续可继续推进：

- 用更稳定的视觉上下文缓存替代临时状态拼接
- 用轻量意图分类模型替代纯规则路由
- 将 YOLO 与 DeepSeek VLM 统一封装为 Agent 工具
- 在场景稳定后逐步扩展到更复杂的家居或实验室任务

本阶段的核心要求不是做大，而是把系统主链路做稳，为后续 Agent 化演进建立可靠基础。